```
"""
game_renderer.py — Unified game drawing for all three missions.

Replaces the 3 separate game_draw() functions in mission1.py, mission2.py,
and mission3.py with a single pipeline-driven renderer.

Architecture:
  1. Each mission registers "draw submissions" into the RenderPipeline
     at the correct layer.
  2. Mission-specific elements are registered by the mission's own
     `populate_pipeline()` method.
  3. The shared elements (bullets, player, particles, HUD, post-FX)
     is drawn by this module — missions don't duplicate it.
  4. `flush()` draws everything in layer order with Y-sorting.

Migration note:
  - mission1.py: replace game_draw() body with:
        renderer = GameRenderer(fcx, fcy)
        renderer.populate_m1()
        renderer.flush()
  - mission2.py: replace game_draw() body with:
        renderer = GameRenderer(fcx, fcy)
        renderer.populate_m2()
        renderer.flush()
  - mission3.py: replace game_draw() body with:
        renderer = GameRenderer(fcx, fcy)
        renderer.populate_m3()
        renderer.flush()
"""

import math
import pygame
from render_pipeline import RenderPipeline
from config import (
    WIDTH, HEIGHT, TILE_SIZE, colour, C_BG_DARK,
    font_terminal_small, C_DARK, C_ACCENT, bullet_glow, MOTION_TRAIL,
    draw_scanlines, draw_vignette,
)
from art_cache import get_blood_decal, get_glow, get_fullscreen_overlay
from state import STATE
from user_settings import SETTINGS
from camera import draw_bullets

class GameRenderer:
    """Unified game renderer using the RenderPipeline.

    Attributes:
        fcx, fcy: Raw camera floats (from update_camera).
        cam_x, cam_y: Integer-snapped camera (prevents tile shimmer).
        pipeline: The RenderPipeline instance for this frame.
    """

    def __init__(self, fcx, fcy, snap_camera=True):
        self.fcx = fcx
        self.fcy = fcy
        # Integer-snapped camera prevents tile/prop seam shimmer when
        # the camera scrolls by fractional pixels. We used raw floats;
        # [illegible] snapped, so snap by default for consistency.
        if snap_camera:
            self.cam_x = int(round(fcx))
            self.cam_y = int(round(fcy))
        else:
            self.cam_x = fcx
            self.cam_y = fcy
        self.pipeline = RenderPipeline()

    # ──────────────────────────────
    # SHARED [illegible] SUBMISSIONS
    # ──────────────────────────────

    def _submit_shared_world(self):
        """Submit draw calls shared by all missions: bullets, thrown bottles,
        enemies, player, particles, casings, pickups, floating texts."""

        cam_x, cam_y = self.cam_x, self.cam_y

        # — Bullets (shared helper from camera.py) —
        self.pipeline.submit(
            'projectiles',
            lambda s: draw_bullets(s, STATE.bullets, cam_x, cam_y, with_glow=True),
        )

        # — Thrown bottles —
        self.pipeline.submit(
            'projectiles',
            lambda s: self._draw_thrown_bottles(s, cam_x, cam_y),
            sort_key=0,
        )

        # — Enemies (Y-sorted by centery) —
        for e in STATE.enemies:
            sort_key = e.rect.centery
            self.pipeline.submit(
                'entities',
                lambda s, _e=e: _e.draw(s, cam_x, cam_y),
                sort_key=sort_key,
            )

        # — Player —
        if STATE.player:
            import death_anim
            if not death_anim.should_skip_player_draw():
[illegible]
```

[illegible]